AI Chatbot with Persistent Memory

Beginner-Friendly Complete Project Documentation (Hinglish)

Project: chatbot-demo-persistance

Version: 0.1.0

Main technologies: Python, Streamlit, LangChain, LangGraph, OpenAI, SQLite

Primary application file: chat_persistance_demo.py

Python requirement: Python 3.11 ya usse newer

---

1. Project ka simple introduction

Ye project ek web-based AI chatbot hai. User browser mein message type karta hai aur chatbot OpenAI model se answer leta hai. Is project ki sabse important capability persistent chat history hai.

Persistent ka simple meaning:

> "App close ya restart hone ke baad bhi purani chats dobara mil sakti hain, kyunki data sirf temporary memory mein nahi, SQLite files mein save hota hai."

App mein user:

- nayi chat create kar sakta hai;
- multiple chats ke beech switch kar sakta hai;
- chat ka naam change kar sakta hai;
- chat metadata delete kar sakta hai;
- purani conversation continue kar sakta hai;
- AI se context-aware follow-up questions pooch sakta hai.

Real-life example

User pehle message bhejta hai:

> "Mera naam Aditi hai aur main Python seekh rahi hoon."

Phir next message bhejta hai:

> "Main kya seekh rahi hoon?"

Same chat ke stored message history ki wajah se model answer de sakta hai:

> "Aap Python seekh rahi hain."

App restart karne ke baad same chat open ki jaaye, tab bhi LangGraph checkpoint se purane messages restore ho sakte hain.

---

2. Learning objectives

Is project ko samajhne ke baad beginner student explain kar paayega:

1. Streamlit app ka execution model kya hai.
2. Chat UI kaise banti hai.
3. LangChain message objects kya hote hain.
4. LangGraph state aur node ka role kya hai.
5. thread_id conversations ko separate kaise rakhta hai.
6. SQLite mein chat metadata aur messages alag kyon store hote hain.
7. Streamlit Session State temporary UI state ko kaise manage karta hai.
8. User message ka end-to-end lifecycle kya hai.
9. Environment variable se OpenAI key kaise load hoti hai.
10. Current code ki security aur production limitations kya hain.

---

3. Sabse pehle big picture

App ko ek restaurant ki tarah imagine karo:

- Streamlit UI waiter hai jo user ka order leta aur result dikhata hai.
- LangGraph workflow manager hai jo decide karta hai order kahan jaana chahiye.

- ChatOpenAI chef hai jo answer banata hai.
- app.db reception register hai jisme chat ka naam aur ID hoti hai.
- checkpoints.db detailed order history hai jisme conversation messages save hote hain.
- Session State waiter ki current shift wali short-term memory hai.

High-level architecture

```
+----------------------+
| User ka Web Browser  |
+----------+-----------+
           |
           | message / button click
           v
+----------------------+
| Streamlit UI         |
| sidebar + chat area  |
+----+-------------+---+
     |             |
     |             | chat_id, title, timestamps
     |             v
     |       +------------+
     |       | app.db     |
     |       | chats table|
     |       +------------+
     |
     | HumanMessage + thread_id
     v
+----------------------+
| LangGraph            |
| START -> chatbot END |
+----------+-----------+
           |
```

```
            | complete message history
            v
+----------------------+
| OpenAI Chat Model    |
| gpt-4o-mini          |
+----------+-----------+
            |
            | AIMessage
            v
+----------------------+
| checkpoints.db       |
| LangGraph state      |
+----------------------+
```

---

4. Technology stack

4.1 Python

Application Python mein likhi gayi hai. Project ko Python >=3.11 chahiye.

4.2 Streamlit

Streamlit Python code se browser UI banata hai. Is project mein ye handle karta hai:

- page configuration;
- sidebar;
- buttons;
- text input;
- chat message display;
- error messages;
- per-browser-session state.

4.3 LangChain

LangChain yahan mainly do kaam karta hai:

- OpenAI chat model ko ChatOpenAI wrapper se call karna;
- messages ko typed objects, jaise HumanMessage aur AIMessage, mein represent karna.

4.4 LangGraph

LangGraph chatbot ka stateful workflow banata hai. Current graph bahut simple hai:

```
START -> chatbot node -> END
```

LangGraph checkpointer conversation state ko SQLite mein persist karta hai.

4.5 OpenAI

ChatOpenAI configured model gpt-4o-mini ko call karta hai. temperature=0 response ko comparatively focused aur repeatable banata hai. Exact output phir bhi guaranteed identical nahi hota.

4.6 SQLite

SQLite lightweight file-based relational database hai. Separate database server run karne ki zarurat nahi hoti.

Project do SQLite files runtime par banata hai:

1. app.db - chat list metadata.
2. checkpoints.db - LangGraph conversation state.

4.7 python-dotenv

.env file ke environment variables load karta hai. OpenAI integration normally OPENAI_API_KEY environment variable expect karti hai.

4.8 uv

Project mein uv.lock present hai. uv fast Python package aur environment manager hai. Lock file exact resolved dependency versions ko repeatable banati hai.

---

5. Project structure

```
chatbot_demo_persistance/
|
|-- chat_persistance_demo.py       # Complete application
|-- pyproject.toml                 # Project metadata and direct dependencies
|-- uv.lock                        # Resolved dependency lock file
|-- .python-version                # Local Python version pin: 3.11
|-- README.md                      # Present hai, lekin currently empty
|-- .env                           # Local secrets/configuration; share nahi karna
|-- .venv/                         # Local Python virtual environment
|-- app.db                         # Runtime par generated chat metadata DB
|-- checkpoints.db                 # Runtime par generated LangGraph DB
|-- PROJECT_DOCUMENTATION_HINGLISH.md
`-- PROJECT_DOCUMENTATION_HINGLISH.pdf
```

Important observation

Current application ka almost poora code ek hi Python file mein hai. Isliye learning ke liye flow follow karna easy hai, lekin large production app mein UI, database, configuration aur AI workflow ko separate modules mein rakhna maintainability ke liye better hota.

Project root mein dedicated .gitignore present nahi hai. Sirf .venv ke andar us environment ka local .gitignore hai; woh root .env, SQLite databases aur Python cache ko protect nahi karta.

Generated database files

Fresh project mein app.db aur checkpoints.db pehle se present hona required nahi hai. Application start hote hi SQLite inhe current working directory mein create kar sakta hai.

---

6. Dependency overview

pyproject.toml ki direct dependencies:

- langchain>=1.3.15
- langchain-openai>=1.5.1
- langgraph>=1.2.11
- langgraph-checkpoint-sqlite>=3.1.1
- python-dotenv>=1.2.2
- streamlit>=1.61.1

uv.lock mein currently resolved important versions:

- LangChain 1.3.15
- LangChain OpenAI 1.5.1
- LangGraph 1.2.11
- LangGraph SQLite Checkpointer 3.1.1
- python-dotenv 1.2.2
- Streamlit 1.61.1

pyproject.toml minimum compatible versions batata hai. uv.lock exact resolved environment batata hai.

---

7. Installation aur setup

7.1 Prerequisites

System par ye hone chahiye:

- Python 3.11 ya newer;
- internet connection;
- valid OpenAI API key;
- uv package manager, recommended.

7.2 Project directory open karo

```
cd /path/to/chatbot_demo_persistance
```

7.3 Dependencies install karo

Recommended:

```
uv sync
```

Ye pyproject.toml aur uv.lock use karke environment prepare karta hai.

7.4 Environment variable configure karo

Project root ki .env file mein:

```
OPENAI_API_KEY=your_real_openai_api_key
```

Rules:

- real key ko documentation, screenshot ya source control mein paste mat karo;
- .env ko Git repository mein commit mat karo;
- key leak ho jaaye to immediately revoke aur rotate karo.

7.5 App run karo

```
uv run streamlit run chat_persistance_demo.py
```

Existing virtual environment manually use karna ho to:

```
source .venv/bin/activate

streamlit run chat_persistance_demo.py
```

Streamlit terminal mein local URL show karega, commonly:

```
http://localhost:8501
```

7.6 App stop karo

Terminal mein:

```
Ctrl + C
```

---

8. Application startup par kya hota hai?

Streamlit script top-to-bottom execute hoti hai. Startup sequence:

1. Python modules import hote hain.
2. load_dotenv() .env variables load karta hai.
3. Streamlit page title, icon aur layout configure hote hain.
4. Custom CSS browser page mein inject hoti hai.
5. app.db connection open hota hai.
6. chats table absent ho to create hoti hai.
7. checkpoints.db connection open hota hai.
8. SqliteSaver checkpointer banta hai.
9. LangGraph State define hota hai.
10. ChatOpenAI client banta hai.
11. chatbot node define hota hai.
12. Graph compile hota hai.
13. Chat metadata Session State mein load hota hai.
14. Chat na ho to initial New Chat create hoti hai.
15. Sidebar aur main chat UI render hoti hai.
16. Selected chat ka LangGraph snapshot load hota hai.
17. Purane messages display hote hain.
18. App user input wait karti hai.

Streamlit rerun ko samjho

Streamlit traditional server-rendered framework jaise sirf ek event handler execute nahi karta. Button click ya input submission par poori script generally dobara top-to-bottom run hoti hai.

Isliye code st.session_state use karta hai. Agar Session State na ho, har rerun par selected chat jaise temporary UI decisions bhool sakte hain.

---

9. Environment configuration

Code startup mein:

```
load_dotenv()
```

Ye current project ki .env values process environment mein load karta hai.

ChatOpenAI constructor mein API key directly pass nahi ki gayi:

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0,
)
```

LangChain OpenAI integration environment se standard OPENAI_API_KEY read karti hai.

Missing key case

Key missing, invalid, expired ya quota-less ho to model invocation fail ho sakti hai. Current UI exception ko:

```
Something went wrong: <actual error>
```

ke form mein show karti hai.

Production mein raw provider error end user ko dikhane ke bajay safe user message aur server-side structured logging better hai.

---

10. Streamlit page aur CSS

Page configuration:

- title: AI Chatbot
- icon: robot emoji
- layout: wide
- sidebar: initially expanded

Custom CSS:

- main container spacing set karti hai;
- sidebar width 280px karti hai;
- chat title/subtitle style karti hai;
- sidebar buttons ko left-aligned aur borderless banati hai;
- hover background deti hai;
- chat message spacing adjust karti hai;
- footer center karti hai.

CSS st.markdown(..., unsafe_allow_html=True) se inject hoti hai. Is case mein HTML static developer-controlled content hai. User message rendering ke liye unsafe_allow_html=True use nahi hua, jo safer behavior hai.

---

11. Database design

Is project mein dual persistence hai.

11.1 app.db: chat metadata

Application manually chats table create karti hai:

```
CREATE TABLE IF NOT EXISTS chats (
    chat_id TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
```

Columns:

- chat_id: unique chat identifier, UUID string.
- title: sidebar mein visible chat name.
- created_at: chat creation timestamp.
- updated_at: latest successful activity ya rename timestamp.

Example row:

```
chat_id     = "bd62a770-7a3d-45d0-a301-8c7857f09d03"

title       = "Explain Python loops"

created_at  = "2026-08-14 17:20:00"

updated_at  = "2026-08-14 17:25:12"
```

11.2 checkpoints.db: conversation state

Ye database directly application-defined simple messages table nahi use karta. LangGraph ka SqliteSaver apni internal checkpoint tables manage karta hai.

Checkpoint mein graph state conversation thread ke according save hoti hai. Main state field:

```
messages: list[BaseMessage]
```

11.3 Do databases kyon?

Responsibilities separate hain:

- sidebar ko fast chat list, titles aur ordering chahiye: app.db;
- LangGraph ko workflow state aur messages restore karne hain: checkpoints.db.

11.4 Dono ko connect kaun karta hai?

Same UUID:

```
app.db chats.chat_id

           ||
```

```
          || same string
          \/
LangGraph configurable.thread_id
```

Code thread_id ko selected chat_id ke equal rakhta hai. Yahi project ka central persistence link hai.

---

12. LangGraph state

State definition:

```
class State(TypedDict):
    messages: Annotated[
        list[BaseMessage],
        add_messages,
    ]
```

TypedDict

Batata hai ki state dictionary ka expected structure kya hai:

```
{
    "messages": [...]
}
```

BaseMessage

LangChain messages ka common parent type hai. Examples:

- HumanMessage
- AIMessage

Annotated

Normal type ke saath extra behavior metadata attach karta hai.

add_messages

Ye reducer new messages ko existing message history ke saath merge/add karta hai. Agar reducer na ho aur plain replacement ho, har invocation purani history overwrite kar sakti thi.

Simple conceptual example:

```
Existing:
[Human("My name is Aditi"), AI("Nice to meet you")]

New input:
[Human("What is my name?")]

Merged state:
[Human("My name is Aditi"),
 AI("Nice to meet you"),
 Human("What is my name?")]
```

Node ka returned AIMessage bhi final state mein add ho jaata hai.

---

13. LLM configuration

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0,
)
```

Model

gpt-4o-mini chat response generate karta hai.

Temperature

temperature=0 ka intention:

- randomness reduce karna;
- focused answer paana;
- repeated requests ko comparatively consistent banana.

Ye factual correctness guarantee nahi karta. AI answer ko critical use cases mein validate karna chahiye.

No system prompt

Current code koi explicit SystemMessage ya system instructions add nahi karta. Isliye bot ka role, tone, safety policy ya domain behavior application level par define nahi hai.

---

14. Chatbot node

Main AI function:

```
def chatbot(state: State) -> State:
    response = llm.invoke(
        state["messages"]
    )

    return {
        "messages": [
            response
        ]
```

```
    }
```

Flow:

1. Node ko current state milti hai.
2. Complete messages history model ko bheji jaati hai.
3. OpenAI se response aata hai.
4. Response AIMessage hota hai.
5. Node ise messages list ke form mein return karta hai.
6. add_messages reducer ise history mein append karta hai.

Important: Function name chatbot hai aur graph node ka registered name bhi "chatbot" hai. Function aur node name related hain, lekin conceptually function executable logic hai aur node graph ka step hai.

---

15. Graph build aur compile

```
START
  |
  v
chatbot
  |
  v
 END
```

Graph setup:

1. StateGraph(State) state schema ke saath builder banata hai.
2. add_node("chatbot", chatbot) AI function register karta hai.
3. START -> chatbot edge entry path banati hai.
4. chatbot -> END edge workflow finish karti hai.
5. builder.compile(checkpointer=checkpointer) executable persistent graph banata hai.

Current graph mein:

- branching nahi hai;

- tool calling nahi hai;
- conditional edges nahi hain;
- human approval step nahi hai;
- multiple agents nahi hain.

LangGraph yahan mainly durable conversation state ke liye valuable hai.

---

16. thread_id: sabse important concept

Graph configuration:

```
config = {
    "configurable": {
        "thread_id": thread_id
    }
}
```

thread_id LangGraph ko batata hai:

> "Kis conversation ki state load aur save karni hai?"

Example:

```
Chat A -> thread_id "111"
Chat B -> thread_id "222"
```

Chat A ke messages Chat B mein nahi aane chahiye, kyunki IDs different hain.

Agar same thread_id reuse ho

LangGraph same conversation continue samjhega aur uski existing history load karega.

Agar har message par new thread_id ho

Bot previous context bhool jaayega, kyunki har message new conversation ban jaayega.

---

17. Database helper functions

17.1 load_chats()

chats table ke saare rows updated_at DESC order mein load karta hai. Return shape:

```
{
    "<chat-id>": {
        "title": "...",
        "thread_id": "<same-chat-id>",
        "created_at": "...",
        "updated_at": "...",
    }
}
```

Most recently updated chat dictionary mein pehle aati hai.

17.2 create_chat_in_db(title="New Chat")

1. uuid.uuid4() se unique ID banata hai.
2. Chat ID aur title insert karta hai.
3. Transaction commit karta hai.
4. New chat ID return karta hai.

Parameterized SQL VALUES (?, ?) use hoti hai. Ye string concatenation se safer hai aur SQL injection risk reduce karti hai.

17.3 update_chat_title(chat_id, title)

Selected chat ka title update karta hai aur updated_at current timestamp par set karta hai.

17.4 update_chat_timestamp(chat_id)

Successful AI graph invocation ke baad latest activity time update karta hai. Isse recent chat sidebar order mein upar aa sakti hai.

17.5 delete_chat_from_db(chat_id)

app.db se chat metadata delete karta hai.

Important limitation: current implementation LangGraph checkpoints ko delete nahi karti. Function comment bolta hai checkpoint cleanup separately handle hogi, lekin current file mein actual checkpoint cleanup call nahi hai.

17.6 get_chat_title(chat_id)

Database se title return karta hai. Row na mile to "New Chat" return karta hai.

Current application flow mein ye helper defined hai, lekin UI logic mein use nahi ho raha.

---

18. Session State

st.session_state current browser session ke across Streamlit reruns mein values preserve karti hai.

Project ki important keys:

- chats: database se loaded metadata dictionary.
- current_chat: selected chat ID.
- menu_chat: jis chat ka rename/delete menu open hai.
- dynamic rename widget keys, jaise rename_<chat_id>.

Session State database nahi hai

Ye distinction yaad rakho:

```
Session State:

- temporary

- current browser session

- UI selection ke liye


SQLite:

- disk persistence

- app restart survive kar sakta hai

- chat records aur conversation state ke liye
```

Initial chat behavior

Session mein chats absent ho to DB se load hota hai.

DB empty ho to:

1. "New Chat" create hoti hai;
2. chats reload hoti hain;
3. new ID current chat ban jaati hai.

DB mein chats already hon aur current_chat absent ho to most recently updated chat select hoti hai.

---

19. Sidebar behavior

Sidebar mein:

- app heading;
- LangGraph + SQLite caption;
- New Chat button;
- saved chat list;
- current chat indicator;
- per-chat menu;
- rename;
- delete;

- cancel.

Current chat icon:

```
🟢 current selected chat
💬 other chat
```

New Chat flow

```
New Chat button
    |
    v
create_new_chat()
    |
    +--> app.db mein row insert
    +--> chats metadata reload
    +--> current_chat new UUID
    |
    v
st.rerun()
```

Switch chat flow

Chat title button click:

1. st.session_state.current_chat selected ID hoti hai.
2. st.rerun() call hota hai.
3. New run mein selected chat ka thread_id config banta hai.
4. graph.get_state(config) uski history load karta hai.
5. UI us conversation ke messages show karti hai.

Rename flow

1. Three-dot button menu ID save karta hai.
2. Text input existing title dikhata hai.

3. Save par whitespace strip hota hai.
4. Empty title ignore hota hai.
5. Valid title DB mein save hota hai.
6. Chat metadata reload hota hai.
7. Menu state remove hoti hai.
8. App rerun hoti hai.

Delete flow

1. Metadata row app.db se delete hoti hai.
2. Remaining chats reload hoti hain.
3. Koi chat bachi ho to first/recent chat select hoti hai.
4. Koi chat na bachi ho to new "New Chat" automatically create hoti hai.

App intentionally at least one metadata chat maintain karti hai.

---

20. Conversation load aur display

Selected chat ke liye:

```
snapshot = graph.get_state(config)
```

Snapshot available ho to:

```
messages = snapshot.values.get("messages", [])
```

Rendering type ke basis par hoti hai:

- HumanMessage -> st.chat_message("user")
- AIMessage -> st.chat_message("assistant")

Unknown message types display nahi hote. Future mein SystemMessage ya ToolMessage state mein aaye to current rendering unhe silently skip karegi.

Empty state

Snapshot ya values absent hon to center mein welcome UI show hoti hai:

```
How can I help you?

Start a conversation with the AI assistant.
```

---

21. User message ka complete end-to-end flow

Example input:

> “Explain recursion in simple words.”

Step 1: Input receive

st.chat_input submitted string deta hai.

Step 2: Input clean

strip() start aur end ke spaces remove karta hai. Empty result ho to processing stop.

Step 3: Automatic title

Agar current title exactly "New Chat" hai, first user message ke first 30 characters title ban jaate hain.

Example:

```
Input:  "Explain recursion in simple words."

Title:  "Explain recursion in simple wo..."
```

Thirty characters se long message mein ... append hota hai, isliye final visible title 33 characters tak ho sakta hai.

Step 4: User message immediately render

UI user message ko model answer se pehle display karti hai.

Step 5: LangGraph invoke

New HumanMessage graph ko selected thread config ke saath diya jaata hai:

```
graph.invoke(
    {
        "messages": [
            HumanMessage(content=user_message)
        ]
    },
    config=config,
)
```

Step 6: Existing state restore

Checkpointer same thread_id ki previous state load karta hai.

Step 7: Message merge

add_messages new human message ko history mein add karta hai.

Step 8: OpenAI call

chatbot node complete message list llm.invoke(...) ko deta hai.

Step 9: AI answer

OpenAI response AIMessage ke roop mein milta hai.

Step 10: Checkpoint save

Compiled graph ka checkpointer updated state ko checkpoints.db mein persist karta hai.

Step 11: Metadata timestamp

Successful graph invocation ke baad app.db mein chat ka updated_at update hota hai.

Step 12: Sidebar metadata reload

Recent ordering refresh karne ke liye chats DB se dobara load hoti hain.

Step 13: AI response display

Result ki last message UI mein assistant response ke form mein show hoti hai.

---

22. Context memory example

First request

```
User: My favorite language is Python.

AI: Great! Python is versatile...
```

Checkpoint state:

```
messages:

1. HumanMessage("My favorite language is Python.")

2. AIMessage("Great! Python is versatile...")
```

Follow-up request

```
User: What is my favorite language?
```

Before model call, merged state conceptually:

```
1. HumanMessage("My favorite language is Python.")

2. AIMessage("Great! Python is versatile...")

3. HumanMessage("What is my favorite language?")
```

AI complete history dekh kar answer de sakta hai:

```
Your favorite language is Python.
```

Separate chat

User new chat create karke poochta hai:

```
What is my favorite language?
```

New thread_id mein old fact available nahi hoga. Model ko previous chat ka context automatically nahi milna chahiye.

---

23. Persistence levels

Project mein teen state levels samjho:

Level 1: Current Python variables

Example:

```
current_chat_id

thread_id

config
```

Ye current script run ke variables hain.

Level 2: Streamlit Session State

Example:

```
st.session_state.current_chat
```

Reruns survive karta hai, lekin permanent database replacement nahi hai.

Level 3: SQLite persistence

app.db aur checkpoints.db disk par data save karte hain aur process restart survive kar sakte hain.

```
shortest life                                    longest life

local variable -> session state -> SQLite files
```

---

24. Chat title rules

New chat default title:

```
New Chat
```

First non-empty message par automatic title:

- first 30 characters;
- longer ho to ...;
- title database mein save;
- current in-memory chat object bhi immediately update.

User baad mein sidebar menu se custom rename kar sakta hai.

Edge case

Agar user manually chat ka naam dobara exact "New Chat" rakh de, next message automatic title logic phir trigger karegi, kyunki code title string compare karta hai; separate “first message

processed" flag use nahi karta.

---

25. Error handling

Current code do main operations par exception catch karta hai.

25.1 Conversation load error

graph.get_state(config) fail ho to:

```
Unable to load conversation: <error>
```

Aur snapshot = None set hota hai.

25.2 Message processing error

Graph/model invocation fail ho to:

```
Something went wrong: <error>
```

Possible reasons:

- missing/invalid OpenAI API key;
- no internet;
- API timeout;
- API quota/rate limit;
- database lock;
- corrupted checkpoint;
- incompatible dependency state.

User bubble graph.invoke() se pehle screen par render hota hai. Agar model/graph call fail ho jaaye, current invocation ka HumanMessage successful checkpoint mein persist nahi hota. Isliye error ke turant baad message screen par dikh sakta hai, lekin refresh/restart par gayab ho sakta hai.

Current gaps

Database CRUD helpers exception handling nahi karti. Rename, create, update ya delete SQL operation fail ho to app run error de sakti hai.

No:

- retry policy;
- structured logging;
- user-friendly error codes;
- provider error sanitization;
- transaction recovery UI;
- observability/metrics.

---

26. Security, privacy aur production readiness

Project learning prototype ke liye useful hai, lekin current form ko multi-user public production deployment ke liye ready assume nahi karna chahiye.

26.1 No authentication

Koi login, user identity ya authorization layer nahi hai.

26.2 Shared chat database

Chats kisi user_id se linked nahi hain. Same app instance ke users potentially same global chat list/data access kar sakte hain.

26.3 Global database connections

Module level par SQLite connections:

```
sqlite3.connect(..., check_same_thread=False)
```

check_same_thread=False connection ko thread restriction se relax karta hai, lekin automatic

thread safety, transaction isolation strategy ya robust concurrency management provide nahi karta.

26.4 Chat delete incomplete hai

Delete operation sirf app.db metadata remove karti hai. checkpoints.db mein associated conversation state remain kar sakti hai. Isliye UI se delete ko complete data erasure nahi maana ja sakta.

26.5 Sensitive prompts external service ko jaate hain

User messages OpenAI API ko send hote hain. Users ko privacy notice, retention policy aur acceptable-use guidance chahiye.

26.6 Secret management

Local .env development ke liye convenient hai. Production mein managed secrets/environment configuration use karni chahiye.

26.7 Raw exception exposure

Actual exception text UI par show hota hai. Ye internal details reveal kar sakta hai.

26.8 No input limits

Code application-level message length limit define nahi karta. Very large prompts cost, latency aur context-window issues create kar sakte hain.

26.9 No moderation/domain guardrails

Explicit system prompt, moderation check, content policy enforcement ya restricted domain behavior absent hai.

---

27. Performance aur scaling considerations

Current behavior

- Every script rerun UI dobara build karti hai.
- Session start par complete chat metadata load hota hai.
- Selected conversation ka complete stored message state render hota hai.
- Complete message history model ko ja sakti hai.
- SQLite local file database use hoti hai.

Long conversation impact

Messages badhne par:

- OpenAI token cost badh sakti hai;
- response latency badh sakti hai;
- context window limit hit ho sakti hai;
- UI rendering slow ho sakti hai;
- checkpoint size grow ho sakta hai.

Many chats impact

Sidebar all chats render karti hai. Pagination, search, archive ya lazy loading nahi hai.

SQLite scaling

SQLite single-instance/light workloads ke liye useful hai. High write concurrency ya horizontally scaled deployment mein shared production database aur intentional connection management chahiye.

---

28. Current limitations summary

1. Saara application logic ek file mein hai.
2. No automated tests.
3. README.md present hai, lekin empty hai.
4. No authentication or per-user ownership.
5. Chat checkpoint deletion implemented nahi hai.

6. No chat list pagination/search.
7. No streaming response; answer complete hone ke baad show hota hai.
8. No system prompt.
9. No tools/RAG/document upload.
10. No token/cost controls.
11. No message editing/regeneration.
12. No export feature.
13. Raw errors end user ko visible hain.
14. Model name code mein hardcoded hai.
15. Database paths current working directory ke relative hain.
16. DB migration/versioning strategy nahi hai.
17. get_chat_title() currently unused hai.
18. Unknown LangChain message types UI mein render nahi hote.
19. Project-root .gitignore absent hai; .env, database aur cache files accidentally version control mein aa sakte hain.
20. Failed LLM invocation ke baad screen par rendered user message refresh par disappear ho sakta hai.

---

29. Troubleshooting guide

Problem: streamlit: command not found

Environment activate nahi hua ya dependencies install nahi hain.

```
uv sync

uv run streamlit run chat_persistance_demo.py
```

Problem: OpenAI authentication error

Check:

- .env project root mein hai;
- variable name exactly OPENAI_API_KEY hai;
- key valid aur active hai;
- key ke around accidental quotes/spaces nahi hain;
- app key update ke baad restart hui hai.

Secret ko terminal output ya chat mein paste mat karo.

Problem: Browser page open nahi ho rahi

Check terminal URL aur port. Common URL:

```
http://localhost:8501
```

Port busy ho to Streamlit alternate port choose kar sakta hai.

Problem: Conversation restore nahi ho rahi

Check:

- app same working directory se run ho rahi hai;
- checkpoints.db delete/move nahi hui;
- selected chat ka chat_id aur graph thread_id same hain;
- filesystem writable hai.

Relative DB paths ki wajah se different directory se command run karne par different database files create ho sakti hain.

Problem: database is locked

Possible reasons:

- same SQLite file par concurrent writes;
- multiple app processes;
- crashed/incomplete transaction;
- filesystem behavior.

Running duplicate app processes check karo. Production fix mein proper connection lifecycle, timeout, WAL evaluation aur stronger storage architecture required ho sakti hai.

Problem: Deleted chat ka data file size reduce nahi hua

Current delete LangGraph checkpoint cleanup nahi karti. SQLite file bhi normal delete ke baad automatically shrink hona guaranteed nahi hai.

Problem: AI previous chat ki information nahi jaanta

Verify same chat selected hai. New chat ka thread_id different hota hai, isliye old chat context intentionally separate hai.

Problem: Chat list stale dikh rahi hai

Session State chats ko cache jaisa hold karti hai. Single-user normal actions reload karte hain, lekin another concurrent session ke updates immediately reflect nahi ho sakte.

---

30. Beginner code-reading roadmap

File ko is order mein padho:

1. Imports.
2. load_dotenv.
3. Streamlit page config.
4. Database connections.
5. State definition.
6. ChatOpenAI.
7. chatbot node.
8. Graph build/compile.
9. Database helper functions.
10. Session initialization.
11. Sidebar actions.
12. Current chat and thread_id.
13. graph.get_state.
14. Message rendering.
15. st.chat_input.
16. graph.invoke.

Is order se “UI first” confusion kam hogi aur data flow clear rahega.

---

31. Important Python concepts used

Imports

External modules ke reusable classes/functions current file mein laate hain.

UUID

```
str(uuid.uuid4())
```

Random-style globally unique chat ID generate karta hai.

Type hints

```
chat_id: str
```

Developer aur tools ko expected type samjhata hai.

Dictionary

Chat metadata key-value form mein store hota hai.

Context manager

```
with st.sidebar:
```

Block ke UI elements sidebar mein place hote hain.

Exception handling

```
try:
    ...
except Exception as error:
```

```
    ...
```

Failure par controlled UI message show karne ki koshish karta hai.

SQL parameter placeholders

```
WHERE chat_id = ?
```

Value separate tuple se pass hoti hai. User value ko SQL string mein directly concatenate nahi karna chahiye.

---

32. Important framework concepts

st.rerun()

Current run stop karke script ko top se immediately rerun karta hai. UI state change ke baad fresh rendering ke liye use hua hai.

st.stop()

Current script execution stop karta hai. Empty stripped message case mein use hua hai.

st.chat_message

User ya assistant role ke visual chat container banata hai.

st.chat_input

Bottom chat composer provide karta hai.

graph.get_state

Thread ke latest persisted graph snapshot ko read karta hai.

graph.invoke

Input state ke saath graph workflow execute karta hai aur updated result return karta hai.

Checkpointer

Graph run ke baad state durable storage mein save karta hai, jisse next invocation/restart par restore ki ja sake.

---

33. Data consistency ko example se samjho

Suppose app.db mein row present hai:

```
chat_id = ABC

title = Python Help
```

Lekin checkpoints.db mein thread ABC absent hai.

Result:

- sidebar chat show karegi;
- conversation empty show hogi.

Opposite case:

checkpoints.db mein thread XYZ present hai, lekin app.db mein metadata row absent hai.

Result:

- normal UI sidebar mein chat show nahi karegi;
- checkpoint orphaned data ban sakta hai.

Current delete behavior second situation create kar sakta hai.

---

34. Manual functional test checklist

Startup

- App bina syntax error ke start hoti hai.
- Sidebar visible hai.
- At least one New Chat visible hai.

First message

- Message UI mein show hota hai.
- AI response aata hai.
- Chat title first 30 characters se update hota hai.

Follow-up context

- First message mein koi fact batao.
- Follow-up mein fact poochho.
- Same chat mein context-based answer verify karo.

Multiple chats

- New chat create karo.
- Different topic par message bhejo.
- Old chat select karo.
- Dono histories separate verify karo.

Rename

- Menu open karo.
- Non-empty title save karo.
- Sidebar title update verify karo.

Delete

- Chat delete karo.
- Metadata list se removal verify karo.
- Last chat delete karne par automatic new chat verify karo.

Restart persistence

- App stop karo.
- App same project directory se restart karo.
- Chat list aur histories restore verify karo.

Failure

- Safe test environment mein invalid key se model error UI behavior verify karo.
- Error ke baad page refresh karke verify karo ki failed turn persistent history ka part nahi bana.
- Real secret log/screenshot mat karo.

---

35. Database inspection commands

SQLite CLI installed ho to metadata inspect:

```
sqlite3 app.db
```

Then:

```
.tables

.schema chats

SELECT chat_id, title, created_at, updated_at

FROM chats

ORDER BY updated_at DESC;

.quit
```

Checkpoint database inspect:

```
sqlite3 checkpoints.db
```

Then:

```
.tables

.quit
```

LangGraph internal tables ko manually edit mat karo. Internal schema library implementation detail hai aur version ke saath change ho sakta hai.

Safe backup

App stopped ho tab simple local backup:

```
cp app.db app.db.backup

cp checkpoints.db checkpoints.db.backup
```

Production backup strategy consistency aur active writes ko account kare.

---

36. Development commands cheat sheet

Install/sync:

```
uv sync
```

Run:

```
uv run streamlit run chat_persistance_demo.py
```

Python syntax check:

```
uv run python -m py_compile chat_persistance_demo.py
```

Stop:

```
Ctrl + C
```

Inspect project metadata:

```
uv tree
```

---

37. Suggested production evolution

Ye current behavior rewrite karne ka instruction nahi, balki architecture learning roadmap hai.

Phase 1: Safety baseline

- .env ignore policy;
- safe errors and logs;
- input size limit;
- explicit configuration validation;
- tests for metadata CRUD and graph thread separation.

Phase 2: User isolation

- authentication;
- user_id ownership;
- authorization on every chat action;
- per-user chat queries;
- complete checkpoint deletion.

Phase 3: Maintainability

- config module;
- database repository/service;
- LangGraph workflow module;
- Streamlit UI module;
- typed return models;
- schema migrations.

Phase 4: Scale and observability

- production database evaluation;
- connection lifecycle;
- metrics, tracing and structured logs;
- rate limiting;
- token/cost monitoring;
- pagination and conversation summarization.

Phase 5: AI quality

- explicit system prompt;
- model configuration through environment;
- safety checks;
- streaming;
- evaluation dataset;
- response quality monitoring.

---

38. Student exercises

Exercise 1: Flow drawing

Paper par user input se AI response tak complete flow draw karo. In labels ko include karo:

- st.chat_input
- HumanMessage
- thread_id
- graph.invoke
- chatbot
- llm.invoke
- AIMessage
- checkpoints.db

Exercise 2: Persistence comparison

Explain karo:

- st.session_state.current_chat
- app.db
- checkpoints.db

teenon ka purpose different kyon hai.

Exercise 3: Title logic

Predict title:

```
How can I understand decorators in Python?
```

Then code ke 30-character rule se actual result verify karo.

Exercise 4: Thread isolation

Do chats create karo:

- Chat A mein bolo favorite color blue hai.
- Chat B mein favorite color poochho.

Observe aur explain karo ki Chat B ko answer kyon pata nahi hona chahiye.

Exercise 5: Orphan checkpoint

Code change kiye bina explain karo ki metadata delete ke baad checkpoint data kyon remain kar sakta hai aur privacy par iska kya impact hai.

---

39. Interview-style questions with answers

Q1. Persistence ka core identifier kya hai?

chat_id, jo LangGraph ke thread_id ke roop mein bhi use hota hai.

Q2. app.db aur checkpoints.db mein difference?

app.db chat list metadata rakhta hai. checkpoints.db LangGraph conversation/workflow state rakhta hai.

Q3. add_messages kyon required hai?

New messages ko old history ke saath merge karne ke liye, taaki conversation context preserve rahe.

Q4. App restart ke baad UI chats kaise find karti hai?

load_chats() app.db ke chats table ko read karta hai.

Q5. Messages kaise restore hote hain?

Selected chat ke thread_id ke saath graph.get_state(config) checkpointer state read karta hai.

Q6. AI response kab metadata timestamp update karta hai?

graph.invoke successful hone ke baad.

Q7. Agar model call fail ho to kya timestamp update hoga?

Nahi. Timestamp update try block mein successful invoke ke baad hota hai.

Q8. Last chat delete karne par kya hota hai?

App automatically ek new "New Chat" metadata row create karti hai.

Q9. Kya delete complete data erasure hai?

Nahi. Current code metadata delete karta hai, LangGraph checkpoints cleanup implement nahi karta.

Q10. Kya app multi-user secure hai?

Current form mein nahi. Authentication aur per-user ownership absent hai.

---

40. Glossary

API: Do software systems ke communication ka contract.

Chat model: Messages input lekar conversational response generate karne wala AI model.

Checkpoint: Workflow state ka saved version.

Context: Previous messages/instructions jo current answer ko influence karte hain.

CRUD: Create, Read, Update, Delete operations.

Database: Structured persistent data storage.

Environment variable: Runtime configuration value, often secrets ke liye.

Graph: Nodes aur edges se represented workflow.

LLM: Large Language Model.

Metadata: Main content ke baare mein supporting data, jaise title aur timestamps.

Node: LangGraph workflow ka executable step.

Reducer: Existing aur new state values ko combine karne ka rule.

Rerun: Streamlit script ka top-to-bottom dobara execute hona.

Session: Ek user/browser interaction period.

Snapshot: Kisi thread ki latest graph state view.

State: Workflow ka current data.

Thread ID: Conversation ko uniquely identify karne wali key.

Token: LLM text processing ki approximate unit.

Transaction: Database changes ka logical unit.

UUID: Highly unique identifier format.

---

41. One-minute project explanation

> “Ye Python Streamlit chatbot hai jo OpenAI ke gpt-4o-mini model ko LangChain wrapper ke through call karta hai. LangGraph ek single chatbot-node workflow manage karta hai. Har chat

> ko UUID milta hai, aur wahi UUID LangGraph thread_id banta hai. app.db sidebar metadata, jaise title aur timestamps, store karta hai; checkpoints.db conversation messages aur graph state persist karta hai. Streamlit Session State selected chat aur menu jaisi temporary UI state rakhti hai. Same thread ke follow-up message par LangGraph previous history restore karke complete context model ko deta hai. Current implementation learning ke liye clear hai, lekin public multi-user production ke liye authentication, user isolation, full deletion, safer errors, tests aur scaling improvements required hain."

---

42. Final recap

Project ka complete mental model:

```
User action
   |
   v
Streamlit rerun
   |
   +--> Session State se selected chat
   |
   +--> app.db se metadata
   |
   v
chat_id == thread_id
   |
   v
LangGraph old checkpoint + new HumanMessage
   |
   v
ChatOpenAI complete history process karta hai
   |
   v
AIMessage return hota hai
```

```
   |
   +--> checkpoints.db mein updated state
   +--> app.db mein updated timestamp
   |
   v
Streamlit response display karti hai
```

Sabse important three points:

1. Streamlit UI manage karta hai.
2. LangGraph conversation state manage karta hai.
3. SQLite app restart ke across data persist karta hai.

Isi combination se project multiple persistent AI chats provide karta hai.

Source-of-truth note

Ye documentation current project files chat_persistance_demo.py, pyproject.toml aur uv.lock ke observed implementation par based hai. Runtime library internals, OpenAI service behavior aur future dependency versions change ho sakte hain. Code aur documentation mein conflict ho to deployed code, database migration state aur official dependency documentation ko verify karna chahiye.